Autonomous Systems Design: Charting a New Discipline

Selma Saidi
Technische Universität Dortmund, 44227 Dortmund, Germany

Dirk Ziegenbein
Robert Bosch GmbH, 71272 Renningen, Germany

Jyotirmoy V. Deshmukh
University of Southern California, Los Angeles, CA 90007 USA

Rolf Ernst
Technische Universität Braunschweig, 38106 Braunschweig, Germany

Editor's notes:
Autonomous systems are becoming pervasive in many domains. This article presents a survey regarding various emerging research directions centered around the broad topic of autonomous systems.
—Partha Pratim Pande, Washington State University

Autonomous systems act independently and solve complex tasks without human intervention, and they are able to learn and adapt to an evolving environment. These capabilities of autonomous systems promise significant benefits and disruption potential in many domains such as mobility, production systems, healthcare, smart home, and smart energy systems. As these domains are classical cyber–physical system (CPS) domains, autonomous systems inherit strict dependability requirements (e.g., safety, security, availability, or real-time) from these systems. Assuring these requirements poses a major challenge on the way to broad market introduction as can be seen, for example, by roadmaps for autonomous driving systems which are currently stretched farther out.

However, many technologies required for building autonomous systems are being researched and developed for years, mostly independently in different communities. In these technologies, significant progress has been made recently, for example, in artificial intelligence (AI) mainly based on machine learning fueled by the availability of data and computation power, in connectivity in terms of bandwidth and reliability enabled by 5G mobile communication, as well as in real-time environment sensing technologies [1].

The design of autonomous systems, however, goes far beyond the integration of the individual components and raises challenges such as designing and verifying system behavior evolving during operation time. This is not covered by traditional system engineering processes. Thus, we consider autonomous systems design not as a discipline concerned with the design of individual components (e.g., designing and training a deep neural network (DNN) for semantic image segmentation) but rather as the design of system architectures, patterns, and mechanisms to provide the autonomous capabilities and to reason about their correctness and safety. For this, an interdisciplinary collaboration between communities such as CPS, AI, SelfAware Computing, Communications, and electronic design automation (EDA) is needed to form a new discipline: autonomous systems design. This article is an attempt to

Digital Object Identifier 10.1109/MDAT.2021.3128434
Date of publication: 16 November 2021; date of current version: 9 February 2022.

Copublished by the IEEE CEDA, IEEE CASS, IEEE SSCS, and TTTC

structure that new discipline into research areas that root in the existing disciplines, yet show the differences and new research directions and their interaction needed for the design of autonomous systems as an engineering discipline.

Autonomous systems

The term "autonomous system" is used to describe systems exhibiting autonomy and autonomicity [2]. Autonomy is the concept of the system being self-governed or self-directed, that is, having flexibility in decision-making to reach its goals based on its knowledge and understanding of the world, its environment, and itself. Autonomicity means that the system is self-sufficient or self-managed, that is, it has the responsibility and capability to maintain its operation, for example, in the presence of failures.

Relation to other domains

While the field of autonomous systems design is currently emerging, the term "autonomous" itself is not new and has been used in various domains such as automation, AI, and CPS. Following [3], we argue that autonomous systems are at the intersection of these domains as depicted in Figure 1.

Automation refers to the ability of a system to independently perform specific tasks in structured and well-defined environments to increase efficiency or productivity. In contrast to an autonomous system having behavioral flexibility, the behavior of an automated system is fully determined by its external input and its internal state (see Automaton).

AI, by definition of the Dartmouth Research Project, is the problem of "making a machine behave in ways that would be called intelligent if a human were so behaving" [4]. This alludes the ability of a system to reason and act intelligently (or like a human) in unknown or uncertain environments and situations by inferring from prior knowledge. Obviously, AI plays an important role in realizing the capabilities of an autonomous system such as understanding the environment and decision-making.

CPSs consist of deeply-intertwined physical components controlled by hardware/software (HW/SW) components. CPS have to fulfill not only functional requirements (often defined over multiple execution steps as known from control theory), but also non-functional requirements such as real-time constraints and dependability requirements, most notably functional safety and security. Typical applications of CPS are automotive, robotics, smart grids, or industrial control. In these application domains, the trend toward autonomy is prevalent such that autonomous systems can be seen as an extension of CPS and typically rely on classical CPS to realize their basic capabilities. Thus, autonomous systems in the sense of autonomous CPS also have to fulfill timing and dependability requirements.

While for classical CPS the environment is well-defined, autonomous systems operate in an environment which is not fully known at design time or evolves during operation time. In this sense, the task to perform and the environment, referred to as operational design domain (ODD) in autonomous vehicles, in which the autonomous system is to be deployed, are critical when designing and verifying autonomous systems. While, on the one hand, no machine (and also no human) is able to competently perform every task in every situation, even a very simple machine can seem to work autonomously if task and environment are sufficiently restricted [5].

Due to the similarities with respect to application domains, dependency on environment, and non-functional requirements, the following description of the main functional blocks of autonomous systems will borrow heavily concepts from the CPS domain.

Functional blocks of autonomous systems

The functional blocks of an autonomous system can be arranged in two overlapping feedback loops as seen in Figure 2: one concerned with the system's interaction with the environment and another one internal to the system. The external

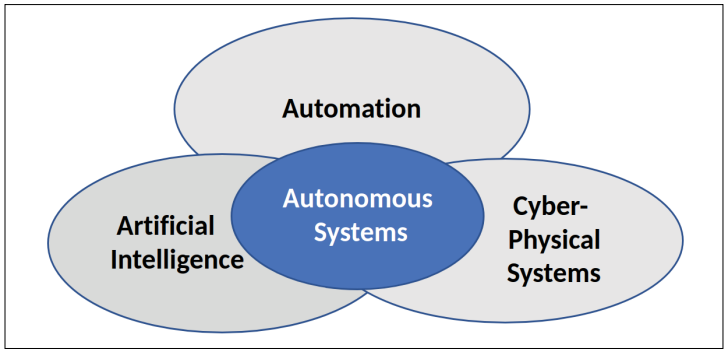

Figure 1. Autonomous systems at the intersection of other domains [3].

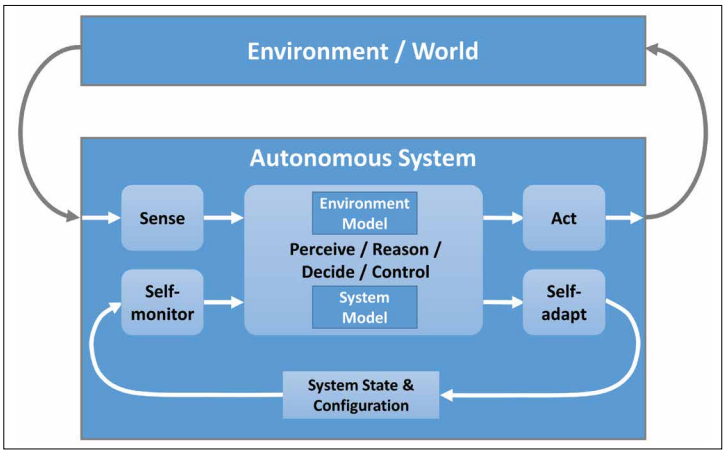

Figure 2. Autonomous system with external and internal feedback loops.

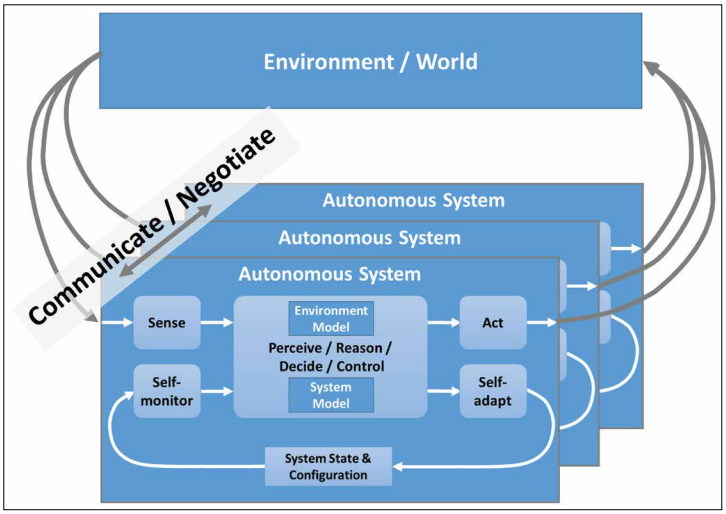

Figure 3. Autonomous system as collaborating intelligent agents.

feedback loop senses the environment, creates an internal environment model denoting the system's understanding and knowledge of the environment, and performs actions which in turn have an effect on the environment. The internal feedback loop monitors the system itself, creates a system model denoting its self-awareness, and performs actions to adapt the system to changes in environment and system state.

Both cycles converge in a central "perceive-reason-decide control" functionality which is responsible for much of the autonomy and AI. This functional block encompasses a variety of different functions operating on different time scales, which are, in general, determined by the dynamics of the processes/activities they control.

- Classical reactive embedded control on typically faster time scales mandated by the time constants of the controlled plant.
- Perception of the environment (e.g., sensor fusion, object classification) and updating the environment model based on sensor inputs on short to medium time scales.
- Task planning (e.g., trajectory generation) on medium time scales mandated by time constants of environment changes (e.g., speed of objects).
- Mission control (e.g., route navigation) on slower time scales mandated by changes to the mission or mission parameters.
- Self-optimization on slower time scales mandated by adaptation and learning processes based on self-evaluation of the effectiveness of the autonomous system over several missions.

We deliberately chose to not further decompose this block as there exist a multitude of different architectures realizing this functionality (see also the following section). One specific architecture for realizing the perceive-reason-decide-control functionality is the *multiagent systems* paradigm [6] where the system behavior is created through the interaction of multiple intelligent agents. In this respect, an autonomous multiagent system can be depicted as in Figure 3 where multiple autonomous agents, which all have their own sensors and actuators and, thus, their own views and models of the environment, communicate and negotiate to reach a common goal.

Another related field which is predominantly dealing with the internal feedback loop in Figure 2 is the field of *self-aware computing systems*. Following the fairly broad definition of Kounev et al. [7], self-aware computing systems learn models about themselves and their environment, reason based on this knowledge, and finally act according to higher-level possibly changing goals. While this definition is fairly similar to our notion of autonomous systems, the focus clearly lies on the ability to reflect about themselves, adapt or optimize their behavior, as well as heal or protect themselves in case of failures or attacks [8].

Autonomous systems design challenges

The current CPS design process is insufficient for autonomous systems design. The V-model-based process that is dominant in industrial design starts with requirements definition and specification proceeding all the way down to detailed implementation, followed by an integration and test process that mirrors the implementation steps. Even though there are many practical variants of that process, the principle of integrating and testing all implemented system parts is prevalent. In addition, safety standards make it a requirement. The benefit of the V-model is full control and verification of the designed system with clear responsibilities, even if distributed in a supply chain.

Autonomous systems design is different. Of course, the autonomous system still relies on hardware and software components which can be implemented as usual. The traditional test and verification methods, however, can only cover the classical basic system functionality (e.g., actuation control, operating system) but not the specific autonomous functionality (e.g., goal-driven decision-making, perception) dealing with uncertainty in the open world. This autonomous behavior is a primary design target and is intended to evolve and adapt during operation time. Therefore, autonomous systems cannot be fully tested and verified in the traditional design process.

Nevertheless, system providers and their designers are still responsible, and eventually liable, for the behavior of an autonomous system. But how can a designer be responsible for a system that lacks predictability due to the evolving context of operation? This lack of predictable behaviors seems to turn critical autonomous systems design into a mission impossible. Thus, in the following sections, we will discuss several challenges and potential approaches to them. The required approaches are summarized in and integrated into an autonomous system design process in the following section and Figure 5.

Assuring and preserving functional safety

Traditional systems safety is governed by standards such as ISO 26262 [9]. They require the development of a safety concept that includes the definition of assurance or safety cases. Such safety cases cover the identification of all relevant safety risks due to system failures and the planned prevention and mitigation actions during design time (e.g., processes, prescribed designs, and test methods) and operation time (e.g., fault detection and reaction). In this sense, assurance cases systematically organize collected evidence (in the form of formal proofs, tests, models and meta-models, design artifacts, formal and informal specifications, and static and dynamic methods for checking system properties) such that it helps designers to argue about the safety and reliability of their designs. In assurance cases for autonomous systems, especially those with learning-enabled components (LECs), in addition to the above artifacts, designers also integrate the data used to guide design decisions as well as the data used for training LECs in the overall assurance case. Such an integration provides traceability from system behavior and specifications to the data and models used during the design process. Here, new techniques for assurance of data-driven models and software components will be required.

The traditional safety standards typically assume a correct and complete system specification. For autonomous systems, it is often impossible to show that the specification is complete, for example, due to the underspecified environment. This asks for additional activities to assure the safety of the intended functionality (SOTIF) in the absence of a fault as mandated by a new standard for road vehicles ISO/PAS 21148 [10]. This standard tries to reduce the probability of functional deficiencies of an autonomous system, for example, due to possible but unforeseen situations or conditions. A particular difficulty in this context is the combinatorial explosion of urban traffic situations which calls for suitable abstractions to argue completeness [11]. The fact that there is a standard however does by no means imply that all challenges in this domain are solved but rather recommends activities and artifacts where suitable methods to perform or deliver them are still subject to further research.

The evolving behavior of autonomous systems poses additional challenges also to established operation time-safety measures such as monitoring. Monitoring typically resorts to multilayer safety architectures where components with simpler behaviors monitor and supervise the main system operation, to interfere and switch to some safety mode in the case of abnormal system behavior (e.g., [12]). Background of such a "safety net" is a systematic distrust

in any complex system function and architecture, assuming that any sufficiently complex system is suspect to contain design errors.

This established approach could be further developed to safeguard against violation of autonomous systems goals and to assure safety constraints. However, further research is necessary. Autonomous system goals and constraints are more complex than current safety goals requiring new monitors and related architectures that support self-reflection. A notion of predictability that focuses on autonomous system goals will be needed to support verification and test in the design process and to guideline diagnosis and systems integration after deployment. We specifically mention systems integration, because coexistence of autonomous systems will require new methods to manage interference, for example, in terms of timing, memory usage, or power consumption [13].

In addition to system safety, ISO 26262 introduces the concept of Safety Element out of Context (SEooC) for the development of hardware and software components that are developed without considering the context of their application in a particular item/vehicle. This is particularly important for enabling component reuse and development productivity for safety-critical systems. SEooC are not developed based on specific system requirements (provided by OEM, suppliers, or technology vendors) but are developed based on multiple assumptions on functionality, application context, external interfaces, and so on. Generally, a distinction between two types of assumptions is considered: Assumptions on the safety requirements for the element itself, for instance, the ASIL for which the component is to be developed. Assumptions on the design context (i.e., system) into which the component will be integrated, for instance, assumptions on safety mechanisms that are provided by the environment to be used by the element. Integration plays then a crucial role to compose the assumed SEooC requirements and build a system meeting the intended functionality and the safety goals. The fact that the environment where the SEooC is to be integrated is not known when designing the component is similar to the underspecified environment of autonomous systems. Thus, the process of formulating and assuring assumptions of SEooC can serve as a blueprint for autonomous systems safety. However, since behavior as well as assumptions will evolve, not all assumptions can be checked at design time but may need to be operationalized as monitors or supervisory components.

An additional challenge is to find the suitable balance between safety and availability, that is, the probability that the system is able to offer its intended use. This is of particular importance for autonomous systems as the human as fallback in the case of failures is taken out of the equation. This means that many system parts, which have been fail-silent in traditional systems, need to be designed as fail-operational, where the system continues to operate in the case of failures.

In the case of safety-critical systems with and without human operators, techniques such as Systems Theoretic Process Analysis (STPA) and the related accident causality model called Systems Theoretic Accident Model and Processes (STAMPs) have seen wide adoption as hazard analysis techniques [14], [15]. These techniques go beyond analyzing the development process or individual component failures by considering the systems operations phase and treating accidents as a control problem and not just as a failure problem. It will be interesting to consider extensions of such hazard analysis techniques in the context of autonomous systems, especially those powered by learning-based algorithms.

(System) architectures for autonomous systems

An architecture in the general computing systems sense refers to a collection of HW/SW components and defines rules on their interaction. Here, we use interaction as a generic term comprising communication and coordination. Autonomous systems are composed of standard components as well as autonomous components which are capable of autonomous actions to achieve their behavioral goals. A major difference with respect to standard components is that the behavior of autonomous components evolves during operation time, for example, to adapt due to changing environment or system conditions.

The HW/SW architecture faces then several challenges to provide capabilities for autonomous components interaction, adaption, and systems coordination to provide the system with the ability to evolve *correctly and safely*. For this, an *infrastructure for autonomy* is required for autonomy supervision

(e.g., preserving assurance for functional safety at operation time) and regulating the interaction and coordination between (autonomous) components in the system [16].

This can be viewed as a multilayered autonomous systems architecture as depicted in Figure 4. We distinguish between autonomous function components (AFCs), usually also referred to as LECs, to implement the intended "perceive, reason, decide, control" functionality and autonomous supervisory components (ASCs) to monitor and adapt the behavior of the AFC as well as to configure the system infrastructure and manage the models about the system itself and its environment (see lower part of Figure 2). In addition, the ASCs are responsible for runtime assurance which encompasses also runtime verification (RV). The idea of multilayered safety architecture for swarms of autonomous drones is also proposed in [17] combining monitoring and navigation coordination for strategic safety (safety-maximizing mission planning), tactical safety (proactive runtime safety maintenance), and active safety (emergency response aiming at mitigating or removing hazards). A multilevel monitoring architecture is also used in functional safety concepts for permissible vehicle acceleration in drive-by-wire systems [12].

Self-monitoring is a HW/SW task that needs fast HW for observing the evolution of autonomous systems, data compression, and interpretation to cope with the enormous amount of collected runtime data. Self-adaptation is a very critical architecture responsibility as it changes the system parameters and even the components and their interactions. Protection is needed against incorrect configurations that might lead to permanent failure or even destructive and hazardous behavior. Model management deals with models storage and distribution. Management functions for updating the models and checking for incorrectness are needed.

Every autonomous component needs to interact with the environment and other components (see upper part of Figures 2 and 3). For this, an autonomous interaction layer is required for model data exchange and model data interaction.

Reliability, security, and trust support of the interaction layer are then a major requirement. Moreover, a notion of interface is needed. The interface defines at design time information/data/states that the component is allowed to disclose at operation time and

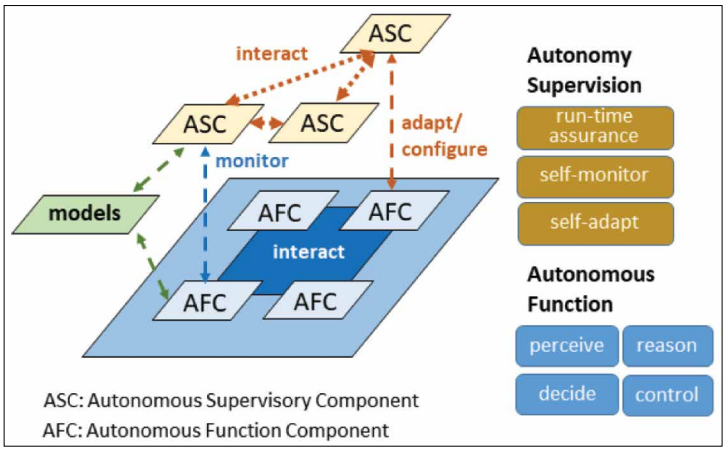

Figure 4. Autonomous systems architecture viewed as multilayered, including autonomous functions (sensing and action omitted) and infrastructure for autonomy.

determines the individual sphere of knowledge that the component can exchange with the environment and other components to contribute the knowledge of the system.

Components interfaces can also include properties and specifications that define conditions (e.g., performance, safety, etc.) on the behavior of the system at design time and its evolution at operation time. These properties can then be verified at operation time by means of monitors. Defining such interfaces for interaction and formulating (parametric) properties that define, for instance, safety guards on the behavior of the system is a major challenge. Properties for individual components can be derived from a global property of the system. Alternatively, individual properties of components can be used to infer global properties on the behavior of the system.

Interaction is also required for supervision. First, self-monitoring depends on the overall system state. Take the example of a degraded vehicle perception (e.g., dirty camera), leading to a lower trust in the "perceive" function output. The lower trust should constrain the permitted decisions (speed, trajectory), immediately affecting the corresponding ASC. Rather than relying on AFC interaction to exchange that data, the corresponding flow of information must occur at the autonomy supervision level. This is a consequence of safety or other reliability requirements: the protective autonomous supervision must not depend on the supervised function; otherwise, the AFCs and their interaction must be certified at the highest safety level [9]. That would either drastically

limit the possible autonomous behavior or require an automated recertification of the autonomous system at operation time.

The example also highlights the difference in interaction requirements. While the AFC interaction targets efficiency, we discussed to this end the infrastructure needed to support autonomy in autonomous components and systems. However, deployment to a HW/SW architecture to meet the above-mentioned requirements for autonomy is left open. How to define appropriate mechanisms and storage infrastructure for managing shared models (on-chip, in the cloud, etc.), parameters, decision, and coordination. A distributed system implementation entails distributed monitoring that will need a monitor communication layer for interpretation and potentially interfere with action.

So far, we have addressed single autonomous systems as depicted in Figure 2. Extending the scope to collaborating intelligent agents, as in Figure 3, adds another level of behavioral complexity. Take the interaction of vehicles at a traffic intersection. Given the speed of traffic, negotiation between vehicles and traffic infrastructure require interaction in the range of a few seconds. The same holds for coordinated agent operation, such as collective perception where sensing of different sources is combined and communicated. In both cases, interaction uses less robust wireless communication. Managing such fast agent interaction over unreliable channels asks for a certain behavioral stability of all involved vehicles. Opening up the internal architecture in Figure 4 to agent interaction appears less suitable, as long as safe system autonomy is still not sufficiently understood. Rather, negotiation and agent interaction could stay at the level of autonomous systems interaction.

Verification of autonomous systems

Verification of autonomous systems presupposes the existence of a *model* of the autonomous system and *high-level specifications* for the system. We do not necessarily preclude real-world physical tests that occur in later stages of development autonomous; however, later stages of development are typically focused on the deployed system and are better served by RV or runtime assurance techniques. RV broadly includes runtime monitoring, verification, and enforcement, and in the context of autonomous

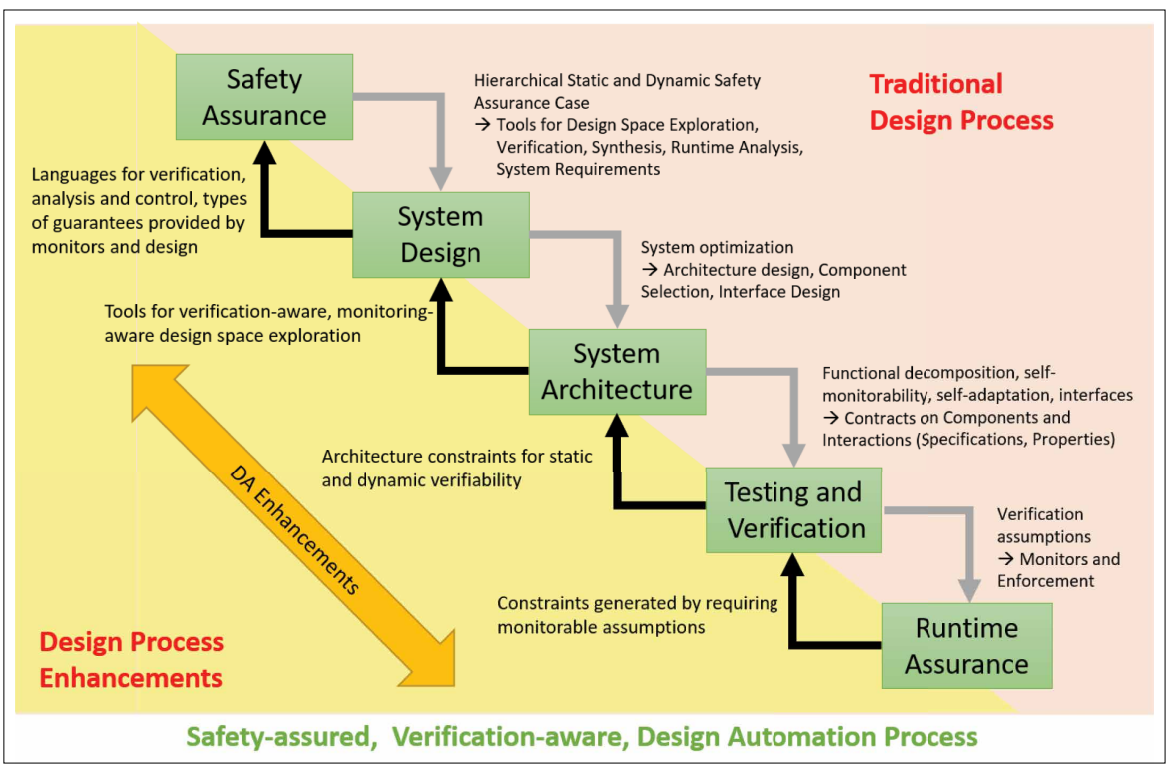

Figure 5. Traditional design process compared with new safety-assured, verification-aware design process required for autonomous system design.

systems, RV is a growing area synthesizing methods for domains such as formal verification, specification monitoring, AI, machine learning, and robotics [18].

Autonomous systems typically consist of many heterogeneous components and compositional verification, that is, ensuring that the composition of the behaviors of individual (possibly verified) components is correct is a significant challenge. The difficulty in autonomous systems is increased because these components may be developed independently and system integration may lead to emergent behavior. A common technique for component-based design of autonomous systems is with the use of design contracts [19]; here, individual components are specified in terms of *guarantees* on the component behavior *predicated on assumptions* on the component's environment. Similarly, we may not be able to anticipate all behaviors of the environment, and we might require properties that are parameterized by assumptions of the environment. Such assume-guarantee contracts and environment assumptions can be used to provide dynamic assurance of a system. Environment and component assumptions become artifacts that can be monitored (and possibly enforced), and similarly component guarantees can also become targets for runtime enforcement [20], [21]. Some autonomous systems explicitly use reflective behavior such as self-awareness and self-adaptability. The verification problem thus has to reason not only about a given system instantiation, but also possible adaptations of the system.

Here, we focus on directions and open questions for verification that is done at design time. The term verification itself needs further elaboration: approaches based on formal methods define verification as either an absolute or probabilistic guarantee that *all possible* system behaviors over finite or infinite time satisfy a certain *property*. Loosely, this means that verification is the process of determining if the design meets high-level specifications or requirements.

There are several approaches for performing *formal verification*. In the class of methods termed *model checking* [22], the verification tool reasons over the (continuous or discrete) state-space of the system. The system states and transitions are represented either explicitly or symbolically, and the model checker checks if any system behavior violates the given specification. While model checking is a fully automatic technique that gives a proof of system correctness, it can be difficult to scale for the size and complexity of typical autonomous systems. Furthermore, for most reasonable autonomous systems, the model checking problem is undecidable—there is no algorithm to model check a generic autonomous system. Hence, a common approach is to build overapproximate abstractions of the system and instead prove the absence of bugs on such tractable abstract systems [23]. While such a procedure can prove the absence of bugs (i.e., produce a proof of correctness), it cannot prove the presence of bugs. This is so because a violation of a specification by the abstract system could be either as a result of the abstraction or the existence of a real bug. An alternative formal approach is based on the use of theorem provers [24], where the user guides a proof assistant to systematically develop a proof-of-system correctness. While theorem proving is an attractive option when we want to consider building assurance cases, usually, it tends to be interactive in nature and requires considerable human ingenuity and effort. Verification of autonomous systems can be performed on just the software (cyber) components of the system, or on a combination of the cyber components in their physical environment. Depending on the models for the physical environment (which can be deterministic or stochastic), the kind of verification methods required can be different. In particular, when the environment is stochastic, techniques such as probabilistic model checking [25] have also been explored.

Static program analysis is a set of approaches related to model checking. While model checking explicitly considers the state transitions generated by the underlying system, static analysis tools interpret the system over abstract states computed directly from the program description and are useful for identifying incorrect programming patterns [26]. In deductive verification approaches (based on the use of automated theorem provers), system correctness is framed as a theorem to be established [24], and various proof strategies are then applied automatically or with human guidance to prove system correctness.

In contrast to the above approaches, techniques such as dynamic analysis and statistical model checking reason about system correctness over actual system executions (or system simulations). In dynamic analysis, we are interested in leveraging information gleaned from a handful of system runs

to make inferences about overall system correctness [27]–[29]. Statistical model checking tools use system executions to (statistically) test the hypothesis that the system behaviors do not satisfy a given specification [30], [31]. Software testing methods are also often used to gain confidence in the system design; here, the developers build test suites consisting of interesting test scenarios (or test cases) and then check if the system exhibits desired behavior for each test case. Testing is complementary to formal verification—typically formal verification methods prove the absence of bugs, while testing proves the presence of bugs. Lighter weight formal methods seek a path between rigorous verification and testing. Falsification [23] is a form of systematic testing that uses an optimization-based, specification-guided approach to drive the system towards failure.

Verification techniques differ based on the kind of model to be verified and the specific property of the model to be verified. Autonomous system design uses many different kinds of models. At a high level, models may differ in the level of fidelity of the model to the actual system, and the level in the design hierarchy being modeled (i.e., whether the model is of the entire system, a subsystem or a specific functional module). In addition to models of the software components of the design target, we often encounter explicit models of the environment of the design target; these encompass physical phenomena, interaction between the system, and its environment and communication between different components of the system. Environment models can be created using causal and acausal modeling frameworks such as Simulink [32] and Modelica [33], respectively, but can also be created in simulation frameworks such as Gazebo [34], Webots [35], AirSim [36], CarSim [37], Carla [38], AimSun [39], Sumo [40], and Open AI Gym [41]. Such environment models themselves can be at different levels of refinement and hierarchy.

To explain the ideas above better, we consider the example of an autonomous or a semi-autonomous car. In such a system, it is common to design the software architecture using a number of diverse functional components, such as those for perception, decision-making, planning, localization, navigation, and low-level control. Perception could further include vision-based perception, radar or LIDAR-based perception, sensor fusion, and so on. Planning could include global route planning, maneuver selection, local path planning, and motion planning. Actuation controllers take reference trajectories from a planner and may implement closed-loop feedback control to ensure trajectory tracking. Each functional module controls a certain aspect of autonomy, and the modules working together control the vehicle to a high-level motion objective. The environment models could include virtual simulation of a real-world driving scenario or actual data (for perception), physical effects such as friction, wind, fuel/battery dynamics (for planning and control modules), and imperfections in communication and sensing (for navigation and localization).

While executable models enable various formal approaches, checking the fidelity of these models to data is itself an important problem; without ensuring validity of the models, any formal analysis of the models is flawed. There have been various approaches for model validation and checking conformance between models; see [42].

While a model is the object of analysis for a verification method, the verification target is a property of the model to be verified, also called a formal requirement or specification. Such specifications are expressed in a mathematically precise and machine-checkable formalism. This includes design-level assertions, properties expressed in various logical formalisms including first-order logic [43], temporal logic [29], automata [44], dynamic logic [24], spatio-temporal logics [45], [46], quality temporal logics for perception [47], [48], and variants that include extensions and combinations of these logics. While not all properties can be described using these formalisms, a broad spectrum of properties of interest can be encoded using these logics.

We can broadly divide these properties to be verified into the following categories.

1. *Functional correctness:* Behavioral models at different levels of design hierarchy (i.e., at the level of the entire system, a subsystem or a module) encode the intended high-level functionality of the hierarchical component at that level. Functional correctness focuses on whether the model satisfies this functional specification. Functional specifications can be specified in terms of input/output behavior of the component, for example, as a desired property of the input/output traces of the component over runs of the system. Such specifications take the form of an *assume-guarantee*

contract: the specification captures assumptions on the inputs to the model that provide a specific guarantee on the outputs of the model.

2. *Functional safety:* Classical approaches to functional safety focus on code metrics such as creating test cases to provide adequate coverage (under a suitable metric). Here, we focus on functional safety properties that reason about the behavior of the model *in conjunction* with its (nondeterministic or stochastic) environment. These properties can be either nondeterministic (requiring all system behaviors to satisfy the property) or statistical (requiring certain statistical or probabilistic properties of the system to be satisfied under the distribution imposed by the stochastic environment).
3. *System performance:* Functional performance properties, that is, how well a system performs its intended task, are related to optimization of design parameters for components in the software stack. Performance properties are treated as soft requirements: these properties are not viewed in terms of satisfaction or violation by the system but as targets for design optimization. This includes aspects such as checking the control performance, responsiveness of the system, optimizing energy consumption, and so on.
4. *Security, privacy, and trust:* Security properties for an autonomous system focus on identifying possible vulnerabilities in the system in the presence of a malicious adversary. Security violations are especially dangerous when they lead to a safety violation, for example, if the adversary gains control of the actuators of the system forcing it in an unsafe configuration. Privacy requirements relate to the possibility of the autonomous system revealing sensitive system information either overtly, or inadvertently when a resourceful adversary is able to learn hidden information from the system's observable behavior. Other non-functional properties of a system include: trustworthiness, ethical decision-making, and fairness. Some of these properties have a direct bearing on the design process for the underlying software components.
5. *Reliability, fault-tolerance, and resilience:* These are typically system-level properties that have to do with component failures. While all of the properties considered so far reason whether the system was designed correctly to have desired behavior, these properties reason about how a system consisting of (possibly) well-designed components responds when one or more components fail. These properties may require system/component models that focus on component failures, their causal impacts, and impact on overall system resilience rather than component/system functionality (e.g., dynamic fault tree models).
6. *Emergence:* Verification approaches for autonomous systems also consider some aspects that are unique to the field of autonomy. As autonomous systems are typically composed by independently designed functional components, when such components are integrated, there may be emergence of previously unseen behavior. This is also true in an autonomous system consisting of independent autonomous agents. Here, the verification problem has to reason not only about desired coordination, co-operation, synchronization, and communication, but also has to reason about emergent behavior.

In summary, we see the following challenges for the successful verification of autonomous systems.

1. *Specification languages:* A hidden problem in any verification approach is the lack of formal specifications that ultimately enable the task of verification. However, there is no industry or academic standard in autonomous system design for such a formal specification language. While other domains such as EDA have embraced formal languages such as PSL [49], such an endeavor for autonomous systems is challenging. Components for perception are fundamentally difficult to specify. Though there are some proposals to reason about correctness of perception systems [47], [48], these efforts are nascent. In decision-making and control systems, various forms of temporal logic such as Linear Temporal Logic and Signal Temporal Logic [29] have been proposed; however the widespread adoption of these languages is yet to happen. Furthermore, autonomous systems may require probabilistic reasoning methods considering the pervasive uncertainty in the environment in which they operate. This would necessitate exploration of specification languages suited for reasoning about probability—a few proposals in this area [50] are also relatively new and have not yet seen widespread application.

2. *Verification for LECs:* The verification problem itself is challenging for several reasons. One of the main reasons is that the use of LECs based on artificial neural networks and other AI-based architectures in software components. Verification for such emerging software models has seen a flurry of recent activity [23]; however, most techniques suffer from scalability and are not yet ready to digest the scale of the LECs used in real-world applications. A related topic is the increasing use of data-driven methods in such LECs. The verification problem for such models not only has to address traditional concerns of safety and functional correctness, but aspects such as the provenance of the data, statistical properties, scenario/scene coverage, bias in the data, and traceability from data to software artifacts will become increasingly important.
3. *Verification methods for self-adaptation:* Autonomous system components need to be self-adapting and self-reflective. Correctness guarantees for such components create upstream requirements on the software architecture and are necessarily parameterized in the form of component assumptions that lead to formal guarantees. Compositional verification remains one of the fundamental challenge problems in verification, and its difficulty is compounded in autonomous systems due to the self-X features that these systems possess. A key challenge is characterizing and reasoning about emergence and verification against potential emergent behavior. An example of this is systems that support life-long learning by combining AI with self-adaptability. Approaches based on coverage metrics have been recently shown to not be effective [51] for such systems.
4. *Monitor synthesis:* As mentioned before, many verification techniques for autonomous systems are capable of providing guarantees only under specific assumptions on their operating environments. For dynamic safety assurance, these assumptions would translate into runtime contracts that simultaneously monitor and enforce correctness of such autonomous systems. Monitor synthesis is a challenge problem for various reasons: monitors need to be non-intrusive (from functionality, overhead, and timing perspective), needs to have architectural and platform support, and the assumptions that we wish to monitor need to be monitorable [52]. When assumptions are statistical in nature, algorithms for detecting a change in the statistical properties of the environment (e.g., covariate shifts) are considered a challenge problem [53].
5. *Runtime enforcement:* Once we have a monitor, enforcing good system behavior (in terms of the properties listed above) is also a challenge. In particular, enforcing properties which relate to functional safety (e.g., maintaining safe distance to other objects in automated driving) is crucial for the release and operation of autonomous systems. Such a runtime enforcement scheme creates new architecture challenges. Once a bad behavior is detected through monitoring, the architecture has to support a number of possible options for the system, for example, runtime repair and continued operation, gradual transition to a safe fallback, transitioning to a simpler control scheme, system shutdown or reset when safe, and so on. Recent work based on shielding systems [20], [54] seems like a promising runtime enforcement methodology, especially if it is able to incorporate domain-specific requirements.

Design automation for autonomous system design

Design automation (DA) must develop formal methods and related tools to enable and control a productive autonomous system design process. One could certainly try to continue with new requirements to the existing DA tool landscape of verification, analysis, and synthesis tools and leave the autonomous system behavior as a completely separate research field that develops its own software methods and mechanisms. DA would then just help to configure and adjust the autonomous systems methods at design time. However, DA could also become an intrinsic part of an autonomous system. For explanation, we can structure the design process into two phases. The "traditional" lab design phase that ends with deployment of the design artifact and an autonomous operation phase that starts with deployment. Many of the established design tasks extend to the autonomous operation phase, such as systems integration, (online) verification, or (re) configuration.

The host of design time DA methods and tools could be revisited investigating their adaptability to the autonomous operation. This is not a completely

new idea, but was already a basis of autonomic computing [8] more than a decade ago, when self-configuration and other "self-X"-capabilities were proposed for large IT system management. There is an active research community pursuing these ideas from self-awareness to runtime methods. A good overview of research into self-aware systems is given in [55], and PROCEEDINGS OF THE IEEE dedicated a recent special issue to the field [56]. This special issue also connects the two fields of self-awareness and multiagent systems [57] that is equally important for autonomous systems. Despite similarities, the focus of self-aware systems research is on the mechanisms of evolution and control using a large variety of architectures.

What has been of less interest so far was a systematic coupling of the two phases in a consecutive process where the tools and methods of a highly developed rigorous design time DA prepare and configure the self-X capabilities of autonomous systems in a coherent DA supported process. There is initial work in that direction, such as for autonomous vehicles [13], but there is far more potential.

1. *DA for the lab design phase:* With the 2-phase structure in mind, we will first address the lab design phase of autonomous systems. A first challenge is to not only formulate concrete behavior and structure of an artifact, but to have adequate means for formulation of behavioral goals and constraints. Suitable languages are needed which provide input to verification, analysis, and control of bounded behavior. The latter will also serve to derive descriptions for monitor synthesis. Inspiration can be taken from the host of work on robot task planning (e.g., [58]) in the past decades.

A second challenge is methods and tools for configuration and design space exploration. New architectures for autonomous systems require new configurable IP blocks and tools for design space exploration. Design space exploration must balance optimization (energy and performance) and sufficient isolation to avoid unwanted interference effects considering the evolving autonomous behavior. Here, either enough budget for evolution of the individual autonomous applications has to be provided at design time or flexible, reconfigurable isolation mechanisms which adapt at operation time need to be deployed.

Monitors support the protection of bounded behavior and the detection of anomalies, such as in the case of destructive emergence. The synthesis of monitors is a HW/SW codesign problem aiming at sufficient flexibility to reconfigure and reprogram the monitor function in response to an evolving autonomous system. Higher system criticalities demand self-diagnosis and self-protection mechanisms that detect and protect against hardware and software errors and security attacks without interfering with regular autonomous system behavior. At the same time, adherence to behavioral conventions, rules, and regulations (e.g., traffic laws) has to be monitored [59]. Observing and bounding complex autonomous system behavior require networks of coordinated monitors. Synthesizing and configuring such networks is a new DA challenge.

2. *DA for autonomous systems operation:* After deployment, a system enters the autonomous operation phase, in which the system behavior autonomously adapts. Thus, DA methods such as design space exploration, failure analysis, and so on need to run at operation time to optimize and safeguard the evolving autonomous system.

If we follow the idea of a coherent design process, then the interface to the lab design phase is an obvious challenge. It must contain all information for autonomous system control and configuration defined in the lab design phase, but can furthermore reuse all data created at design time, such as models which can be used to set up and maintain a self-image as a basis of self-X methods (self-configuration, self-awareness, etc.) or data sets for self-learning. Self-diagnosis will also profit from such models. The availability of a comprehensive self-image opens new DA opportunities for critical systems, such as automated failure analysis (FMEA, FTA), dependency analysis, and online verification. Currently, failure analysis is a design time method, severely limiting behavioral dynamics of autonomous systems. The availability of automated failure analysis will open up the behavioral space for autonomous systems.

Like in lab design, DA at operation time depends on the data quality. Using, augmenting, and verifying design data in autonomous operation is, therefore, a crucial condition for the application of DA methods in autonomous operation.

Putting it all together—The autonomous system design process

This article approached autonomous systems design as an engineering discipline targeting comprehensive critical and mixed-critical systems that integrate one or several autonomous applications. We started with the observation that the current CPS design processes is insufficient for designing such systems and must, therefore, be revised. However, we are not free to drop the CPS design process with its benefits in terms of efficiency, control, and clear assignment of responsibilities. We rather suggest a design process enhancement that keeps compatibility with the "traditional" industrial design process and mitigates the risk of transition.

Figure 5 outlines the proposed design process enhancement. For convenience of presentation, we unfolded the V-model where testing and sign-off verification (including systems integration) is on the right branch. Safety assurance in the conceptual phase and runtime assurance in the operation phase are typical steps that distinguish the design of (mixed) critical systems and are elaborated in the related safety standards. This traditional design process is augmented by further tasks and tools targeting the new autonomous systems architecture as described in this article. The more fundamental change is in the design process itself. The design process enhancements of Figure 5 introduce feedback that is needed to control the self-X capabilities of an autonomous system in its operation phase. As an example, runtime assurance uses assumptions that must be monitored in operation. The ability to monitor such assumptions must be guaranteed in testing and verification. Testing and verification, in turn, is augmented by dynamic self-verification which is only executed in operation. The augmented capabilities must be assured by systems features developed in the system architecture design phase. This feedback sequence continues all the way back to the safety assurance phase where the self-X capabilities and the constraints of the operation phase can, now, be taken into account on the basis of a comprehensive autonomous systems evaluation. This evaluation includes all dependencies of self-X mechanisms and their implementations as determined in the enhanced design process. The tool environment must be extended accordingly, as indicated in Figure 5. It has to be noted that while the challenges described in previous section are addressed by the proposed process, many of the individual techniques and methods to fully solve the challenges are still a matter of research.

An autonomous system has flexibility in decision-making to reach its goals based on its knowledge and understanding of the world, its context, and itself. Designing autonomous systems with traditional design methods either overly constrains the system behavior or falls short in controlling its potential behavior, which is not acceptable critical applications.

This article tried to chart the discipline, starting with general concepts and requirements, then deriving the challenges of assurance, architectures, verification, and DA. It follows the principle of a controlled autonomy with a lab design phase that defines the autonomous behavior together with change and control mechanisms that are active in a systems operation phase, thereby effectively moving part of the traditional design process to the field. The resulting design process must include resilience and assurance, not only for an autonomous individual, but also in autonomous systems collaboration. Assurance needs online methods and tools with architectural support for online supervision in operation addressing both traditional functional safety and SOTIF. The close relation of user-controlled lab design and self-governed system autonomy leads to new design task dependencies that are unknown in a state-of-the-art design processes. These dependencies define the permissible design space of controlled autonomy. The more we understand and exploit that relation, the wider we can open the controlled design autonomy. Therefore, autonomous systems design is not only an interesting research topic, but is of great importance for an efficient and responsible industrial design of autonomous systems. ■

References

[1] R. Dumitrescu et al., *Studie, 'Autonome Systeme' (in German)*. Expertenkommission Forschung und Innovation (EFI), Feb. 2018.

[2] C. Rouff, *Autonomous and Autonomic Systems: With Applications to NASA Intelligent Spacecraft Operations and Exploration Systems (NASA Monographs in Systems and Software Engineering)*. Berlin, Germany: Springer-Verlag, 2007.

[3] L. G. Shattuck. (Mar. 2015). *Transitioning to Autonomy: A Human Systems Integration Perspective*. [Online]. Available: https://human-factors.arc.nasa.gov/workshop/autonomy/download/presentations/Shaddock20.pdf

[4] J. McCarthy et al., "A proposal for the dartmouth summer research project on artificial intelligence, August 31, 1955," *AI Mag.*, vol. 27, no. 4, pp. 12–14, 2006.

[5] J. M. Bradshaw et al., "The seven deadly myths of autonomous systems," *IEEE Intell. Syst.*, vol. 28, no. 3, pp. 54–61, May 2013.

[6] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Hoboken, NJ, USA: Wiley, 2009.

[7] S. Kounev et al., *Self-Aware Computing Systems*, 1st ed. New York, NY, USA: Springer-Verlag, 2017.

[8] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, Jan. 2003.

[9] *Road Vehicles—Functional Safety*, Standard ISO 26262:2018, 2018.

[10] *Road Vehicles—Safety of the Intended Functionality*, Standard ISO/PAS 21448:2019, 2019.

[11] M. Butz et al., "SOCA: Domain analysis for highly automated driving systems," in *Proc. 23rd IEEE Int. Conf. Intell. Transp. Syst. (ITSC)*, Sep. 2020, pp. 1–6.

[12] EGAS Working Group, *Standardized E-Gas Monitoring Concept for Gasoline and Diesel Engine Control Units—Version 6.0*, 2015.

[13] M. Mostl et al., "Platform-centric self-awareness as a key enabler for controlling changes in CPS," *Proc. IEEE*, vol. 106, no. 9, pp. 1543–1567, Sep. 2018.

[14] T. Ishimatsu et al., "Hazard analysis of complex spacecraft using systems-theoretic process analysis," *J. Spacecraft Rockets*, vol. 51, no. 2, pp. 509–522, Mar. 2014.

[15] I. Friedberg et al., "STPA-SafeSec: Safety and security analysis for cyber-physical systems," *J. Inf. Secur. Appl.*, vol. 34, pp. 183–196, Jun. 2017.

[16] *Broad Agency Announcement—Assured Autonomy*, Defense Adv. Res. Projects Agency, Arlington County, VA, USA, Aug. 2017.

[17] I. Vistbakka, E. Troubitsyna, and A. Majd, "Multi-layered safety architecture of autonomous systems: Formalising coordination perspective," in *Proc. IEEE 19th Int. Symp. High Assurance Syst. Eng. (HASE)*, Jan. 2019, pp. 58–65.

[18] J. Deshmukh and D. Ničković, *Runtime Verification: 20th International Conference, RV 2020, Los Angeles, CA, USA, October 6-9, 2020, Proceedings* (Lecture Notes in Computer Science Series). New York, NY, USA: Springer-Verlag, 2020.

[19] P. Nuzzo et al., "Chase: Contract-based requirement engineering for cyber-physical system design," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, 2018, pp. 839–844.

[20] B. Konighofer et al., "Shield synthesis for reinforcement learning," in *Proc. Int. Symp. Leveraging Appl. Formal Methods*, 2020, pp. 290–306.

[21] D. de Niz, B. Andersson, and G. Moreno, "Safety enforcement for the verification of autonomous systems," *Proc. SPIE, Auton. Syst., Sensors, Vehicles, Secur., Internet Everything*, vol. 10643, May 2018, Art. no. 1064303.

[22] C. Baier and J.-P. Katoen, *Principles of Model Checking*. Cambridge, MA, USA: MIT Press, 2008.

[23] J. V. Deshmukh and S. Sankaranarayanan, "Formal techniques for verification and testing of cyber-physical systems," in *Design Automation of Cyber-Physical Systems*. Cham, Switzerland: Springer, 2019, pp. 69–105.

[24] A. Platzer, *Logical Analysis of Hybrid Systems: Proving Theorems for Complex Dynamics*. New York, NY, USA: Springer-Verlag, 2010.

[25] M. Kwiatkowska, G. Norman, and D. Parker, "Probabilistic model checking: Advances and applications," in *Formal System Verification*. Cham, Switzerland: Springer-Verlag, 2018, pp. 73–121.

[26] X. Zheng et al., "Perceptions on the state of the art in verification and validation in cyber-physical systems," *IEEE Syst. J.*, vol. 11, no. 4, pp. 2614–2627, Dec. 2017.

[27] A. Donzé and O. Maler, "Systematic simulation using sensitivity analysis," in *Proc. Int. Workshop Hybrid Syst., Comput. Control*. Berlin, Germany: Springer-Verlag, 2007, pp. 174–189.

[28] C. Fan et al., "DryVR: Data-driven verification and compositional reasoning for automotive systems," in *Proc. Int. Conf. Comput. Aided Verification*. Cham, Switzerland: Springer-Verlag, 2017, pp. 441–461.

[29] E. Bartocci et al., "Specification-based monitoring of cyber-physical systems: A survey on theory, tools and applications," in *Lectures on Runtime Verification*. Cham, Switzerland: Springer-Verlag, 2018, pp. 135–175.

[30] K. G. Larsen and A. Legay, "Statistical model checking: Past, present, and future," in *Proc. Int. Symp. Leveraging Appl. Formal Methods*. Berlin, Germany: Springer-Verlag, 2014, pp. 135–142.

[31] M. Zarei, Y. Wang, and M. Pajic, "Statistical verification of learning-based cyber-physical systems," in *Proc.*

23rd Int. Conf. Hybrid Syst., Comput. Control, Apr. 2020, pp. 1–7.

[32] *Simulink R2020a*, MathWorks, Natick, MA, USA, 2020.

[33] P. Fritzson, *Introduction to Modeling and Simulation of Technical and Physical Systems With Modelica*. Hoboken, NJ, USA: Wiley, 2011.

[34] N. P. Koenig and A. Howard, "Design and use paradigms for Gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, vol. 3, Sep. 2004, pp. 2149–2154.

[35] O. Michel, "Cyberbotics Ltd. Webots: Professional mobile robot simulation," *Int. J. Adv. Robot. Syst.*, vol. 1, no. 1, p. 5, 2004.

[36] S. Shah et al., "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," in *Field and Service Robotics*. Cham, Switzerland: Springer-Verlag, 2018, pp. 621–635.

[37] T. Kinjawadekar et al., "Vehicle dynamics modeling and validation of the 2003 ford expedition with ESC using CarSim," SAE Tech. Paper 2009-01-0452, 2009.

[38] A. Dosovitskiy et al., "CARLA: An open urban driving simulator," 2017, *arXiv:1711.03938*.

[39] J. Casas et al., "Traffic simulation with Aimsun," in *Fundamentals of Traffic Simulation*. New York, NY, USA: Springer-Verlag, 2010, pp. 173–232.

[40] D. Krajzewicz, "Traffic simulation with SUMO—Simulation of urban mobility," in *Fundamentals of Traffic Simulation*. New York, NY, USA: Springer-Verlag, 2010, pp. 269–293.

[41] G. Brockman et al., "OpenAI gym," 2016, *arXiv:1606.01540*.

[42] H. Roehm et al., "Model conformance for cyber-physical systems: A survey," *ACM Trans. Cyber-Phys. Syst.*, vol. 3, no. 3, pp. 1–26, Aug. 2019.

[43] A. Bakhirkin et al., "The first-order logic of signals: keynote," in *Proc. Int. Conf. Embedded Softw.*, 2018, pp. 1–10.

[44] K. G. Larsen, "Verification and performance analysis of embedded and cyber-physical systems using UPPAAL," in *Proc. 2nd Int. Conf. Model-Driven Eng. Softw. Develop. (MODELSWARD)*, 2014, p. IS-11.

[45] E. Bartocci et al., "Moonlight: A lightweight tool for monitoring spatiotemporal properties," in *Proc. Int. Conf. Runtime Verification*, 2020, pp. 417–428.

[46] L. Bortolussi et al., "Automatic translation of spatio-temporal logics to streaming-based monitoring applications for IoT-equipped autonomous agents," in *Proc. 6th Int. Workshop Middleware Appl. Internet Things*, 2019, pp. 7–12.

[47] A. Dokhanchi et al., "Evaluating perception systems for autonomous vehicles using quality temporal logic," in *Proc. Int. Conf. Runtime Verification*, 2018, pp. 409–416.

[48] A. Balakrishnan et al., "Specifying and evaluating quality metrics for vision-based perception systems," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, 2019, pp. 1433–1438.

[49] C. Eisner and D. Fisman, *A Practical Introduction to PSL*. New York, NY, USA: Springer-Verlag, 2006.

[50] P. Kyriakis, J. V. Deshmukh, and P. Bogdan, "Specification mining and robust design under uncertainty: A stochastic temporal logic approach," *ACM Trans. Embedded Comput. Syst.*, vol. 18, no. 5s, pp. 1–21, Oct. 2019.

[51] S. Abrecht et al., "Revisiting neuron coverage and its application to test generation," in *Proc. 39th Int. Conf. Comput. Saf., Rel., Secur. (SafeComp)*, 2020, pp. 289–301.

[52] T. A. Henzinger and N. E. Saraç, "Monitorability under assumptions," in *Proc. Int. Conf. Runtime Verification*, 2020, pp. 3–18.

[53] M. Sugiyama, M. Krauledat, and K.-R. Müller, "Covariate shift adaptation by importance weighted cross validation," *J. Mach. Learn. Res.*, vol. 8, pp. 985–1005, May 2007.

[54] M. Wu et al., "Shield synthesis for real: Enforcing safety in cyber-physical systems," in *Proc. Formal Methods Comput. Aided Design (FMCAD)*, 2019, pp. 129–137.

[55] P. R. Lewis et al., *Self-Aware Computing Systems*. New York, NY, USA: Springer-Verlag, 2016.

[56] N. Dutt et al., "Self-awareness for autonomous systems," *Proc. IEEE*, vol. 108, no. 7, pp. 971–975, Jul. 2020.

[57] L. A. Dennis and M. Fisher, "Verifiable self-aware agent-based autonomous systems," *Proc. IEEE*, vol. 108, no. 7, pp. 1011–1026, Jul. 2020.

[58] R. Barták, M. A. Salido, and F. Rossi, "New trends in constraint satisfaction, planning, and scheduling: A survey," *Knowl. Eng. Rev.*, vol. 25, no. 3, pp. 249–279, Sep. 2010.

[59] A. Rizaldi and M. Althoff, "Formalising traffic rules for accountability of autonomous vehicles," in *Proc. IEEE 18th Int. Conf. Intell. Transp. Syst.*, Sep. 2015, pp. 1658–1665.

Selma Saidi is a Professor of embedded systems with the Technische Universität Dortmund, Dortmund, Germany. Her research involves the design, implementation and validation of innovative intelligent embedded systems. Key aspects are the development of novel hardware and software design methods for embedded and autonomous systems where performance, predictability and self-adaptability play an important role. Saidi has a PhD in computer science from the University of Grenoble, Grenoble, France, conducted together with STMicroelectronics.

Dirk Ziegenbein is a Chief Expert for open context systems engineering and leads a research group developing methods and technologies for software systems engineering at Bosch Corporate Research, Stuttgart, Germany.

Jyotirmoy V. Deshmukh is an Assistant Professor with the Department of Computer Science, Viterbi School of Engineering, University of Southern California, Los Angeles, CA, USA. His research interests are in the design, verification, testing, and analysis of cyber–physical systems (CPSs), machine learning and AI approaches in robotics and control, and software engineering.

Rolf Ernst is a Professor with the Technische Universität Braunschweig, Braunschweig, Germany, where he is the Chair of the Institute of Computer and Network Engineering. His research activities include embedded system design and cyber–physical systems (CPSs), with a focus on critical systems, such as in automotive and avionics systems. He is a Fellow of IEEE.

■ Direct questions and comments about this article to Selma Saidi, Technische Universität Dortmund, 44227 Dortmund, Germany; selma.saidi@tu-dortmund.de.